2-2 线程

- 1. 线程的基本概念
  - 定义
    - **单线程进程**：一个进程只有一个执行点，即一个程序计数器。
    - **多线程进程**：一个进程有多个执行点，即多个逻辑程序计数器。
    - **线程**：进程中的一个执行流，是处理机调度和分派的基本单位。
  - 线程的关键特点
    - **共享地址空间**：同一进程内的多个线程共享进程地址空间。
    - **共享数据**：线程可访问相同的全局变量、堆、打开文件等。
    - **独立执行点**：每个线程有自己的程序计数器和寄存器状态。
    - **独立栈**：每个线程有自己的栈。
    - **依附进程**：线程没有独立地址空间，也不独立拥有资源。
  - 线程共享什么
    - **地址空间。**
    - **程序代码。**
    - **全局变量。**
    - **打开文件。**
    - **子进程。**
    - **堆。**
    - **进程拥有的其他资源。**
  - 线程独有
    - **程序计数器 PC。**
    - **寄存器。**
    - **栈。**
    - **线程状态。**
    - **线程标识符。**
- 2. 线程状态与转换
  - 三种基本状态
    - **就绪态**：线程已具备执行条件，只差 CPU。
    - **执行态**：线程获得 CPU，正在运行。
    - **阻塞态**：线程因某事件受阻，处于暂停状态。
  - 状态转换
    - **就绪 -> 执行**：线程被调度，获得 CPU。
    - **执行 -> 就绪**：时间片用完，或被更高优先级线程抢占。
    - **执行 -> 阻塞**：线程发出 I/O 请求或等待某事件。
    - **阻塞 -> 就绪**：I/O 完成或等待事件发生。
  - 与进程状态的关系
    - **状态名称类似**：线程也有就绪、执行、阻塞。
    - **调度粒度更细**：多线程系统中，OS 或线程库可以调度线程。
    - **同进程内影响不同**：一个线程阻塞是否影响其他线程，取决于线程实现方式。
- 3. TCB
  - 基本概念
    - **TCB**：Thread Control Block，线程控制块。
    - **作用**：记录线程当前情况和线程管理所需信息。
    - **地位**：类似进程中的 PCB，但面向线程。
  - TCB 中的信息
    - **线程标识符**：每个线程唯一的标识符。
    - **寄存器内容**：PC、PSW、通用寄存器等。
    - **线程执行状态**：描述线程当前处于就绪、执行或阻塞等状态。
    - **优先级**：描述线程执行优先程度。
    - **线程专有存储区**：用于线程切换时保存现场和线程相关信息。
    - **信号屏蔽**：对某些信号加以屏蔽。
    - **堆栈指针**：包括指向用户栈和内核栈的指针。
- 4. 线程控制
  - 线程创建
    - **初始线程**：用户程序启动时，通常只有一个初始化线程在执行。
    - **创建方式**：初始化线程调用线程创建函数创建新线程。
    - **返回结果**：线程创建函数返回一个线程标识符。
    - **效果**：进程内部出现新的执行流。
  - 线程终止
    - **终止方式**：线程调用相应函数执行终止操作。
    - **资源释放不一定立即发生**：多数 OS 中，线程终止后不会马上释放所有资源。
    - **分离函数**：进程中其他线程执行分离函数后，终止线程才与资源分离。
    - **资源复用**：资源分离后才能被其他线程利用。
    - **特殊情况**：已终止但尚未释放资源的线程，仍可被需要它的线程调用以恢复运行。
- 5. 用户级线程 ULT
  - 基本概念
    - **ULT**：User-Level Thread，用户级线程。
    - **实现位置**：整个线程实现部分放在用户空间。
    - **内核视角**：内核看到的是一个单线程进程。
    - **管理者**：线程库和用户程序中的线程调度代码。
  - 优点
    - **切换开销小**：线程切换在用户空间完成，无需用户态/内核态切换。
    - **调度灵活**：每个进程可以选择自己的线程调度算法。
    - **与 OS 无关**：ULT 管理代码在用户程序中，可在不支持线程机制的 OS 上实现。
    - **实现效率高**：线程管理不必频繁进入内核。
  - 缺点
    - **内核无法感知线程**：内核调度对象仍是整个进程。
    - **无法发挥多处理器优势**：一个进程一次只能获得一个 CPU，进程内只有一个线程能执行。
    - **阻塞影响大**：进程内某个线程发起 I/O 系统调用，可能导致整个进程所有线程阻塞。
    - **资源竞争范围小**：线程只能在进程内部竞争资源。
- 6. 内核支持线程 KST
  - 基本概念
    - **KST**：Kernel-Supported Thread，内核支持线程，也称内核级线程。
    - **实现位置**：在内核支持下运行。
    - **内核视角**：操作系统可以感知线程。
    - **管理者**：内核负责线程创建、调度、阻塞和切换。
  - 优点
    - **内核可识别线程**：调度对象是内核级线程。
    - **并行能力强**：多处理机系统中，可给不同内核级线程分配 CPU。
    - **阻塞影响小**：一个线程阻塞时，内核可以调度同进程其他线程或其他进程线程运行。
    - **全系统竞争资源**：线程可以在全系统范围内竞争处理机资源。
  - 缺点
    - **切换开销大**：每次线程切换都需要 CPU 状态转换，开销较大。
    - **内存占用较大**：TCB 存放在内核空间。
    - **管理复杂**：线程管理由内核完成，需要更多内核机制支持。
- 7. ULT vs KST
  - 对比表

对比项	用户级线程 ULT	内核支持线程 KST
实现位置	用户空间	内核空间
内核是否感知	不感知	感知
调度者	开发者/线程库	OS 内核
切换开销	小	大
阻塞影响	一个线程阻塞可能导致整个进程阻塞	一个线程阻塞可调度其他线程
多处理器并行	难以发挥	可以发挥
资源竞争	进程内部	全系统
OS 依赖	低	高

- 8. 多对一模型
  - 基本概念
    - **映射关系**：多个用户级线程映射到一个内核级线程。
    - **线程管理**：在用户空间完成。
    - **内核视角**：用户级线程对 OS 不可见。
  - 优点
    - **效率高**：线程管理在用户空间进行。
    - **切换快**：线程切换不需要进入内核。
    - **实现简单**：对内核支持要求低。
  - 缺点
    - **阻塞问题**：一个线程使用内核服务被阻塞，整个进程都会被阻塞。
    - **不能真正并行**：多个用户级线程只对应一个内核级线程，难以同时在多个 CPU 上执行。
- 9. 一对一模型
  - 基本概念
    - **映射关系**：每个用户级线程都映射到一个内核级线程。
    - **内核视角**：内核能看到每个线程。
    - **调度方式**：每个线程都可被内核独立调度。
  - 优点
    - **并发能力强**：一个线程阻塞时，其他线程可继续运行。
    - **适合多处理器**：多个线程可分配到多个 CPU 上并行执行。
    - **调度灵活**：内核可按线程粒度调度。
  - 缺点
    - **开销大**：每个用户级线程都需要一个内核级线程对应。
    - **资源占用多**：TCB 和内核线程结构需要占用内核资源。
    - **线程数量受限**：系统能支持的线程数量可能受内核资源限制。
- 10. 多对多模型
  - 基本概念
    - **映射关系**：`n` 个用户级线程映射到 `m` 个内核级线程，`m <= n`。
    - **设计思想**：结合多对一模型和一对一模型的优点。
    - **目标**：在效率和并发能力之间折中。
  - 优点
    - **比多对一并发更强**：多个内核级线程可并行运行。
    - **比一对一开销更小**：不需要每个用户级线程都对应一个内核级线程。
    - **灵活性高**：可根据需要调整用户级线程与内核级线程的映射。
  - 缺点
    - **实现复杂**：需要协调用户级线程和内核级线程。
    - **调度复杂**：用户空间和内核空间都可能参与调度。

by joshuji 由 幕布 发布